

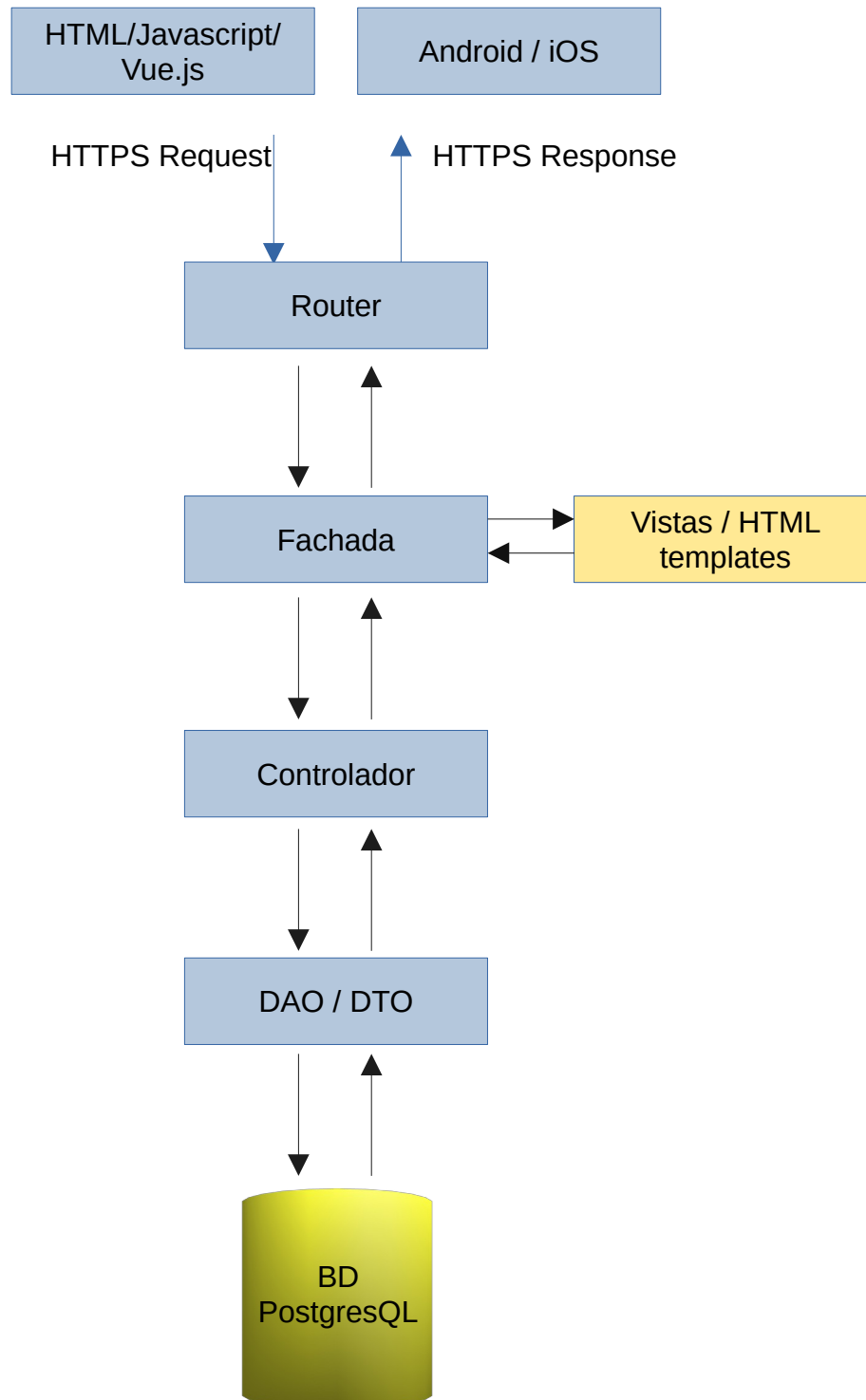
Documento de desarrollo

Tecnologías utilizadas

Nombre	Descripción
Base de datos	Postgres 15.6 en adelante
Backend	Go 1.19 en adelante
Frontend	HTML + Vue.js

Arquitectura del sistema

Componentes generales



Detalle Componentes generales

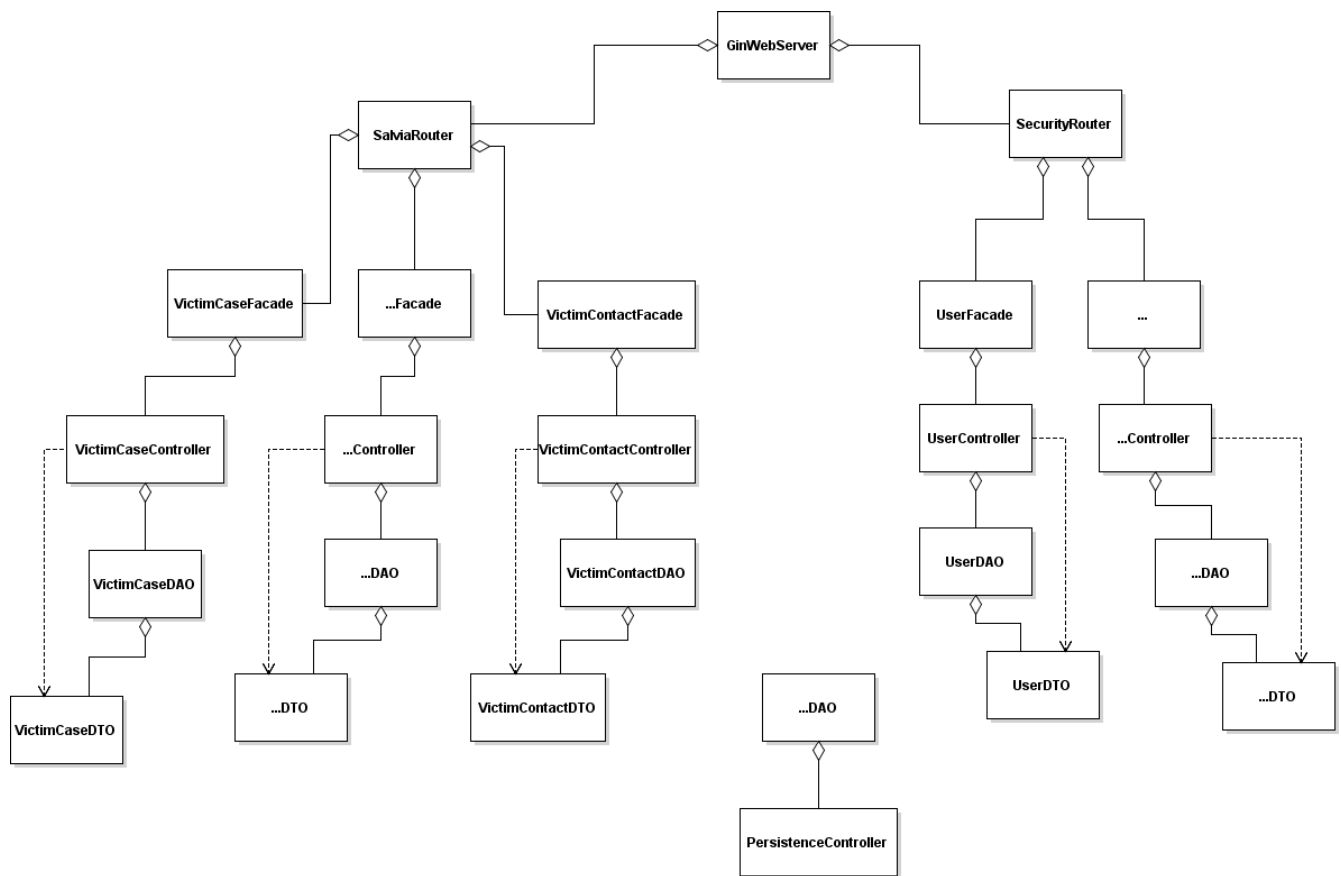


Diagrama de clases UML para el módulo Seguridad

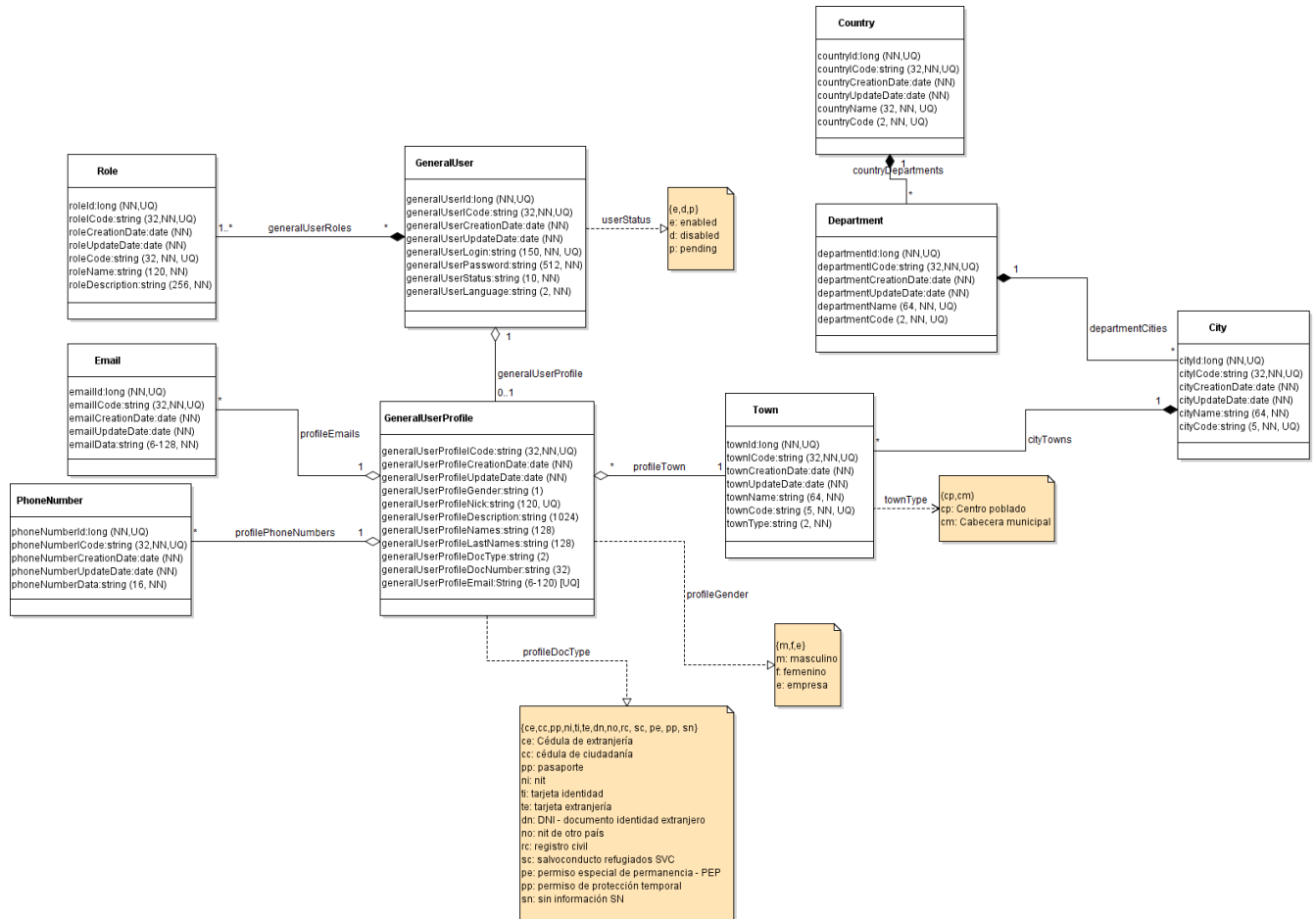
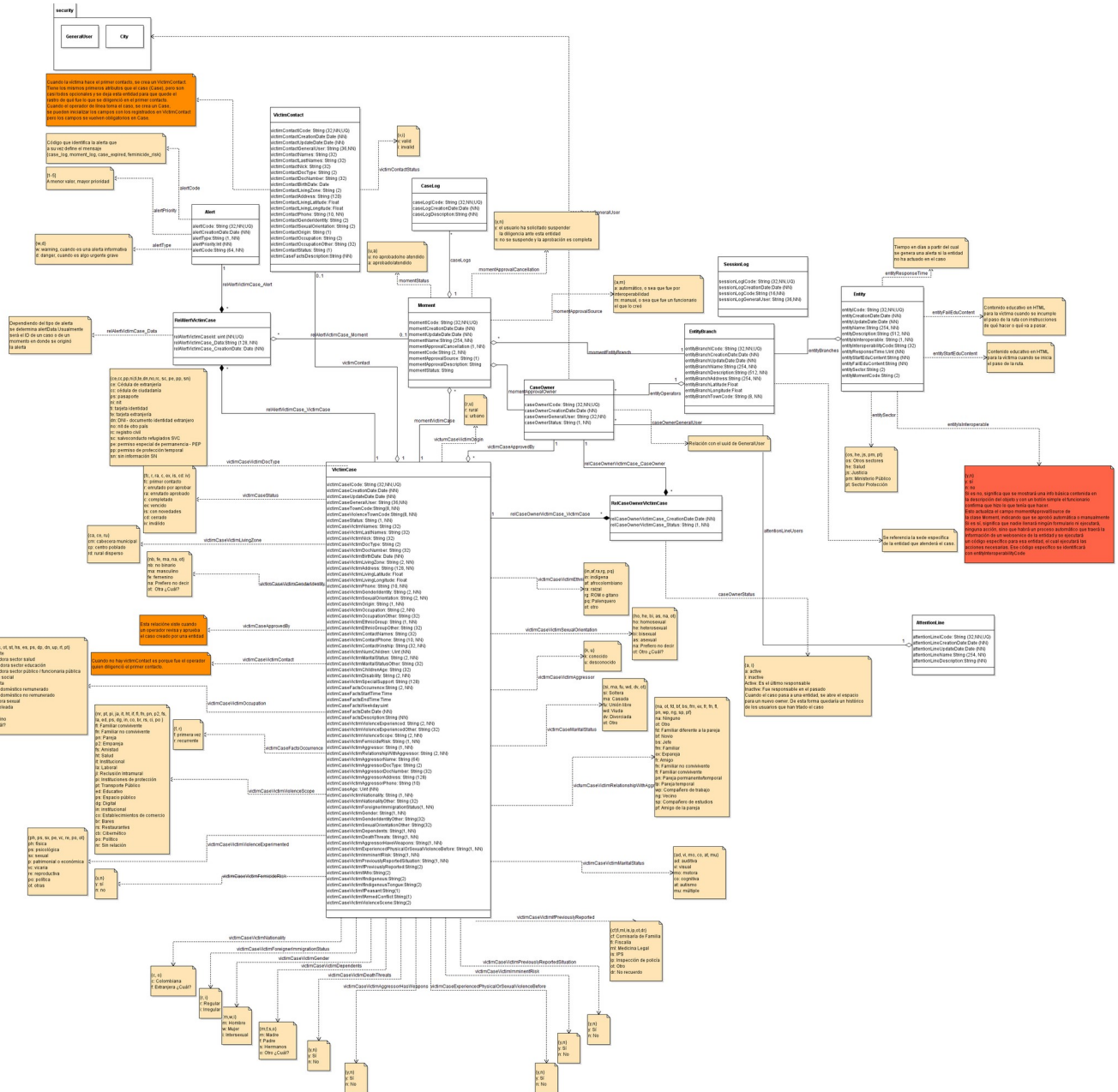


Diagrama de clases UML para el módulo Salvia



Descripción de la arquitectura

A continuación se hará un recorrido por la arquitectura desde la parte más interna del backend hasta la externa terminando en el frontend.

DAO/DTO

Se utiliza ambas metodologías, tanto la Data Access Object como la Data Transfer Object.

DAO: encargada de darle persistencia a los objetos. El proyecto contiene la carpeta llamada daos en donde se almacenan todos los DAOs, uno por cada clase. Por ejemplo la clase VictimCase, tiene su respectivo VictimCaseDAO. Allí están todos los métodos CRUD para VictimCase. Allí también está definido internamente el VictimCaseDTO que representa la clase con sus atributos.

DTO: representa la clase con sus atributos y este objeto es el que se utiliza para transportar la información por las diferentes capas. Cada atributo tiene su definición en JSON, por lo tanto en capas superiores, cuando se requiere su serialización, el DTO se convierte en su respectiva representación JSON.

Todos los DAO tienen esta estructura al comienzo del archivo:

```
var {
    VictimCaseEntityName string = "VictimCase"
    VictimCaseJSONName    string = "victimCase"
    VictimCaseDBName      string = "victim_case"
    VictimCaseDBScheme    string = "salvia"

    //Atributos relacionados con las validaciones -----|

    //Campos que vienen como string del JSON. El booleano indica si son strings en el modelo o no (como en el caso de una fecha)
    VictimCaseFieldDefinitions map[string]utils.FieldDefinition = map[string]utils.FieldDefinition{
        "VictimCaseId": {Name: "VictimCaseId", DBName: "victim_case_id", Alias: "", ModelType: "uint", MinSize: 0, MaxSize: 0, Required: true},
        "VictimCaseICode": {Name: "VictimCaseICode", DBName: "victim_case_i_code", Alias: "", ModelType: "string", MinSize: 0, MaxSize: 36, Required: true},
        "VictimCaseCreationDate": {Name: "VictimCaseCreationDate", DBName: "victim_case_creation_date", Alias: "", ModelType: "datetime", MinSize: 0, MaxSize: 0, Required: true},
        "VictimCaseUpdateDate": {Name: "VictimCaseUpdateDate", DBName: "victim_case_update_date", Alias: "", ModelType: "datetime", MinSize: 0, MaxSize: 0, Required: true},
        "VictimCaseStatus": {Name: "VictimCaseStatus", DBName: "victim_case_satatus", Alias: "", ModelType: "string", MinSize: 1, MaxSize: 2, Required: true},
        "VictimCaseGeneralUser": {Name: "VictimCaseGeneralUser", DBName: "victim_case_general_user", Alias: "", ModelType: "string", MinSize: 32, MaxSize: 32, Required: true},
        "VictimCaseNames": {Name: "VictimCaseNames", DBName: "victim_case_victim_names", Alias: "", ModelType: "string", MinSize: 3, MaxSize: 32, Required: true},
        "VictimCaseLastNames": {Name: "VictimCaseLastNames", DBName: "victim_case_victim_last_names", Alias: "", ModelType: "string", MinSize: 3, MaxSize: 32, Required: true},
        "VictimCaseNick": {Name: "VictimCaseNick", DBName: "victim_case_victim_nick", Alias: "", ModelType: "string", MinSize: 3, MaxSize: 32, Required: true},
        "VictimCaseDocType": {Name: "VictimCaseDocType", DBName: "victim_case_victim_doc_type", Alias: "", ModelType: "string", MinSize: 2, MaxSize: 2, Required: true},
        "VictimCaseDocNumber": {Name: "VictimCaseDocNumber", DBName: "victim_case_victim_doc_number", Alias: "", ModelType: "string", MinSize: 4, MaxSize: 4, Required: true},
        "VictimCaseBirthDate": {Name: "VictimCaseBirthDate", DBName: "victim_case_victim_birth_date", Alias: "", ModelType: "date", MinSize: 0, MaxSize: 0, Required: true},
        "VictimCaseTownCode": {Name: "VictimCaseTownCode", DBName: "victim_case_victim_town_code", Alias: "", ModelType: "string", MinSize: 2, MaxSize: 2, Required: true},
        "VictimCaseAddress": {Name: "VictimCaseAddress", DBName: "victim_case_victim_address", Alias: "", ModelType: "string", MinSize: 3, MaxSize: 3, Required: true},
        "VictimCaseLivingLatitude": {Name: "VictimCaseLivingLatitude", DBName: "victim_case_victim_living_latitude", Alias: "", ModelType: "float", MinSize: 0, MaxSize: 0, Required: true},
        "VictimCaseLivingLongitude": {Name: "VictimCaseLivingLongitude", DBName: "victim_case_victim_living_longitude", Alias: "", ModelType: "float", MinSize: 0, MaxSize: 0, Required: true},
        "VictimCasePhone": {Name: "VictimCasePhone", DBName: "victim_case_victim_phone", Alias: "", ModelType: "string", MinSize: 6, MaxSize: 16, Required: true}
```

Las primeras variables tienen una estructura que se repite en todos los DAOs en donde **lo anaranjado es el nombre de la clase y siempre es diferente dependiendo la clase**, y **lo morado es el significado de la variable** que siempre tendrá la misma forma para todas las clases,

VictimCaseEntityName: contiene el nombre de la clase

VictimCaseJSONName: contiene el nombre de la clase cuando se serializa como JSON, con el fin de poder expresar cualquier nombre; puede coincidir con el nombre interno de la clase o podría usarse cualquier otro por temas de seguridad.

VictimCaseDBName: contiene el nombre de la tabla en la base de datos correspondiente a la clase actual

VictimCaseDBScheme: contiene el esquema al que pertenece la tabla en la base de datos

Luego aparece la lista de todos los atributos de la clase y su mapeo con la base de datos, en donde hay unos que son universales para todas las clases como por ejemplo:

VictimCaseId: pk de la tabla en base de datos

VictimCaseIcode: pk secundaria interna aleatoria para uso público y evitar que externamente se tenga acceso a la pk principal

VictimCaseCreationDate: Fecha de creación del objeto

VictimCaseUpdateDate: Fecha de última actualización del objeto

Veamos un ejemplo de cómo se definió **VictimCaseIcode**:

```
"VictimCaseIcode": {Name: "VictimCaseIcode", DBName: "victim_case_i_code", Alias: "", ModelType: "string", MinSize: 8, MaxSize: 36, Required: true}
```

Name: Nombre del atributo, se repite pero es para efectos de búsquedas posteriores

DBName: Nombre del atributo en la base de datos

Alias: Opcionalmente se puede elegir un alias por defecto para ese atributo

ModelType: tipo de dato del atributo para validaciones.

MinSize: tamaño mínimo del contenido. Si es numérico, se evalúa su valor, si es string se evalúa la cantidad de caracteres.

MaxSize: tamaño máximo del contenido. Si es numérico, se evalúa su valor, si es string se evalúa la cantidad de caracteres.

Required: true, si es obligatorio o false si es opcional.

Luego se define el DTO, que contiene la misma lista de atributos, pero esta vez se definen no en el sentido de la base de datos sino del modelo de negocio:

```
type VictimCaseDTO struct {
    VictimCaseId          uint64      `json:"-"`
    VictimCaseIcode       string      `json:"icode"`
    VictimCaseCreationDate time.Time   `json:"creationDate"`
    VictimCaseUpdateDate  time.Time   `json:"updateDate"`
    VictimCaseStatus      string      `json:"status"`
    VictimCaseGeneralUser string      `json:"user"`
    VictimCaseNames       string      `json:"names"`
    VictimCaseLastNames   string      `json:"lastNames"`
    VictimCaseNick        string      `json:"nick"`
    VictimCaseDocType     string      `json:"docType"`
    VictimCaseDocNumber   string      `json:"docNumber"`
    VictimCaseBirthDate   time.Time   `json:"birthDate"`
}
```

Encontramos el nombre del atributo, el tipo y la última columna define el nombre que tendrá cuando se serialice

a JSON. Cuando aparece "-", significa que ese atributo no se representará en el JSON y queda oculto, como en el caso de la llave primaria.

En esa lista de atributos pueden haber algunos adicionales que no están en la tabla en base de datos, pero que se necesita para poder operar con la información. En estos casos se indica con un comentario:

```
VictimCaseApprovedBy      security_daos.GeneralUserDTO    `json:"approvedBy"`
//Campos de formulario que no hacen parte del modelo o no directamente en la BD -----
VictimCaseEntityBranches  map[string]map[string]map[string][]string `json:"entityBranchesByMomentIdx"`
VictimCaseVictimContactICode string `json:"victimContactIcode"`
VictimCaseNewUser         security_daos.GeneralUserDTO    `json:"newUser"`
VictimCaseDepartment      security_daos.DepartmentDTO    `json:"department"`
VictimCaseCity            security_daos.CityDTO          `json:"city"`
VictimCaseTown            security_daos.TownDTO          `json:"town"`
VictimCaseMoments         []MomentDTO                    `json:"moments"`
VictimCaseTownLatitude    float64                        `json:"townLatitude"`
VictimCaseTownLongitude   float64                        `json:"townLongitude"`
}
```

Debido a restricciones con el lenguaje GO que no permite campos nulos en variables, y dado que en base de datos pueden haber campos nulos, entonces se tuvo que utilizar un tipo de datos especial para base de datos y se creó otro tipo de DTO pero orientado a postgres:

```
type VictimCasePgDB struct {
    VictimCaseId          sql.NullInt64
    VictimCaseICode       sql.NullString
    VictimCaseCreationDate sql.NullTime
    VictimCaseUpdateDate  sql.NullTime
    VictimCaseStatus      sql.NullString
    VictimCaseGeneralUser sql.NullString
    VictimCaseNames       sql.NullString
    VictimCaseLastNames   sql.NullString
    VictimCaseNick        sql.NullString
    VictimCaseDocType     sql.NullString
    VictimCaseDocNumber   sql.NullString
    VictimCaseBirthDate   sql.NullTime
    VictimCaseTownCode    sql.NullString
    VictimCaseAddress     sql.NullString
    VictimCaseLivingLatitude sql.NullFloat64
}
```

Como puede verse en la anterior imagen, es la misma lista de atributos pero con un tipo de datos especial de sql. Y en el nombre, en lugar de DTO, se le puso PgDB. El objetivo es que cuando hay datos nulos en base de datos se utiliza este objeto para traerlos, y luego con un método de este tipo de datos se puede convertir en DTO normal para seguir con el flujo de eventos. Para eso se implementó este método:


```
func (obj *VictimCasePgDB) ToDTO() VictimCaseDTO {
    var dto VictimCaseDTO

    if obj.VictimCaseId.Valid {
        dto.VictimCaseId = uint64(obj.VictimCaseId.Int64)
    }

    if obj.VictimCaseICode.Valid {
        dto.VictimCaseICode = obj.VictimCaseICode.String
    }

    if obj.VictimCaseCreationDate.Valid {
        dto.VictimCaseCreationDate = obj.VictimCaseCreationDate.Time
    }
}
```

En donde se verifica si es válido, entonces se asigna el valor al dto. De lo contrario no se hace nada, lo que significa que atributo en el DTO queda con los valores por defecto. Esta es la forma de respresentarlo en GO.

Luego viene la función MarshalJSON que es parte de una interfaz de GO, se invoca automáticamente cuando se intenta serializar el objeto. Esta función sólo es necesaria cuando se requiere hacer alguna operación manual, como por ejemplo ponerle un formato a las fechas a la hora de convertir el bjeeto a JSON, por lo tanto sólo se especifican esos atributos que se modificarán:

```
func (vcd VictimCaseDTO) MarshalJSON() ([]byte, error) {
    type Alias VictimCaseDTO // Crea un alias para evitar recursión infinita

    // Formatea la hora como deseas (por ejemplo, "2006-01-02 15:04:05")

    // Crea una estructura anónima con el formato deseado
    return json.Marshal(&struct {
        *Alias
        VictimCaseBirthDate    string `json:"birthDate"`
        VictimCaseCreationDate  string `json:"creationDate"`
        VictimCaseUpdateDate    string `json:"updateDate"`
        VictimCaseFactsStartTime string `json:"factsStartTime"`
        VictimCaseFactsEndTime  string `json:"factsEndTime"`
    }{
        Alias:      (*Alias>(&vcd),
        VictimCaseBirthDate:    vcd.VictimCaseBirthDate.Format(common_config.DateTime.DATE_FORMAT),
        VictimCaseCreationDate: vcd.VictimCaseCreationDate.Format(common_config.DateTime.DATE_FORMAT),
        VictimCaseUpdateDate:   vcd.VictimCaseUpdateDate.Format(common_config.DateTime.DATE_FORMAT),
        VictimCaseFactsStartTime: vcd.VictimCaseFactsStartTime.Format(common_config.DateTime.TIME_FORMAT),
        VictimCaseFactsEndTime: vcd.VictimCaseFactsEndTime.Format(common_config.DateTime.TIME_FORMAT),
    })
}
```

Luego vienen los métodos que representan el CRUD y toda la lógica básica de persistencia. Aquí no hay lógica de negocio, sólo el acto básico de consultar o almacenar el objeto en base de datos.

Finalmente, viene el método que inicializa el objeto con lo básico y automático:

```
// Algunas utilidades
func SetVictimCaseDefaults(victimCase *VictimCaseDTO, action string, s utils.CommonSession) {

    switch action {
    case common_dao.SQL_INSERT:
        victimCase.VictimCaseCreationDate = time.Now()
        victimCase.VictimCaseUpdateDate = time.Now()
        victimCase.VictimCaseICode = uuid.New().String()
        if s.CurrentRole == "op" {
            victimCase.VictimCaseStatus = "ra"
        } else if s.CurrentRole == "et" {
            victimCase.VictimCaseStatus = "r"
        }

    case common_dao.SQL_UPDATE:
        victimCase.VictimCaseUpdateDate = time.Now()
    }
}
```

Así como `SetVictimCaseDefaults`, todos los DAO tiene su respectivo método para los valores por defecto. Se pasa por parámetro la acción, que indica si se inicializa para un Insert en BD o para un Update, ya que en cada caso se inicializan los atributos de diferente manera. Se eligió crear el uuid en el código y no en BD para que no hubiese una dependencia de funciones o plugins en la base de datos. Se envía el objeto como apuntador para que los cambios se reflejen en el objeto original y se pasa el atributo `s` que es de tipo **CommonSession**, que contiene los datos de la sesión ya que en este caso se necesitan. Pero en otros DAO este atributo no es requerido.

Controladores

Los controladores son los encargados de la lógica de negocio. Así como para cada clase hay un DAO, también hay un controlador. Sin embargo, un DAO no puede comunicarse con otro DAO, pero un controlador sí puede comunicarse con otros DAO u otros controladores.

Los controladores tienen una estructura básica con los CRUD y funciones complementarias. De resaltar, tienen unas líneas comunes e importantes. Continuando con el mismo ejemplo:

```
func SetVictimCase(dataInput string, module string, s utils.CommonSession, dbClientConfig db.DBClientConfig, dbServerConfig db.DBServerConfig) (int, string) {

    var err error = nil

    var connID string
    defer db.ReleaseConnection(&connID)
    var collectedErrors map[string]map[string]string = map[string]map[string]string{}

    //Se crean los DTO requeridos
    var vCase salvia_daos.VictimCaseDTO = salvia_daos.VictimCaseDTO{}
    var victimContact salvia_daos.VictimContactDTO = salvia_daos.VictimContactDTO{}
    var approvedBy salvia_daos.CaseOwnerDTO = salvia_daos.CaseOwnerDTO{}

    var dtoMap map[string]interface{} = nil
    //Se ponen los valores por defecto
    salvia_daos.SetVictimCaseDefaults(&vCase, common_dao.SQL_INSERT, s)
```

Todos los métodos tienen el mismo contrato en el que retornan el código de éxito o error y el contenido:

SetVictimCase(dataInput string, module string, s utils.CommonSession, dbClientConfig db.DBClientConfig, dbServerConfig db.DBServerConfig) (int, string)

dataInput: viene el formulario serializado, o el cuerpo del mensaje del webservice en texto plano.

Module: indica el módulo en el que se está trabajando. Actualmente Salvia sólo contiene 2 módulos: uno llamado “salvia” que contiene toda la funcionalidad del reporte de casos y otro llamado “security” que es el módulo de usuarios y autenticación. A futuro podrían haber más módulos.

s: es la sesión en curso que contiene unos atributos básicos del usuario en sesión, como el idioma, el rol, y si tiene una ciudad o entidad asociados, etc

dbClientConfig: son los datos de conexión a la base de datos. Se pasa siempre en caso de que en algún momento hubiese información distribuida en diferentes bases de datos, entonces para cada request se podría especificar una base de datos diferente.

dbServerConfig: configuración básica del servidor, que en este caso sólo contiene el número máximo de conexiones abiertas a la base de datos

Luego en el cuerpo de la función, encontramos algunas líneas de interés:

var connID string: representa un ID de conexión a la base de datos. Este ID se pasa en cada request al DAO para que utilice la misma conexión durante todo el flujo de eventos. Cuando hay transacciones en curso es cuando esto importa más, ya que a la hora de hacer un commit o rollback, todo debe estar amparado en el mismo ID para que la transacción sea consistente.

var checkFields map[string]bool: contiene la lista de los atributos que se van a operar y si serán obligatorios o no. Aunque en el DAO están definidos los campos obligatorios, existe la posibilidad de que hayan campos que no sean obligatorios en BD pero que a nivel de lógica de negocio se deban poner obligatorios en ciertos casos o para ciertos usuarios. Entonces en este atributo se puede establecer qué campos se van a evaluar y si son requeridos.

```
47     var rawDto interface{} = utils.GetDTOMap(dataInput, salvia_daos.VictimCaseJSONName, common_
48
49     //Verificamos que Los datos vengan coh la esatruectura inicial que queremos: string:interfac
50     switch rawDto.(type) {
51     case map[string]interface{}:
52         dtoMap = rawDto.(map[string]interface{})
53
54         utils.ValidateJSONInput(&vCase, dtoMap, salvia_daos.VictimCaseJSONName, salvia_daos.Vic
55         //-----
56
57         switch dtoMap[salvia_daos.VictimCaseJSONName].(type) {
58         case map[string]interface{}:
59             victimCase := dtoMap[salvia_daos.VictimCaseJSONName].(map[string]interface{})
60
```

A partir de esta parte del código, se empieza a convertir el dataInput serializado en un objeto de tipo mapa, y en la función **utils.ValidateJSONInput** se intenta convertir ese mapa en un DTO para empezar a trabajar con este tipo de objeto.

Fachadas

Las fachadas son las encargadas de recibir el HTTP Request, obtener las variables que vienen en las URL y allí se verifican los permisos:

```
if !utils.CheckPermission(salvia_config.PermissionsByRole, "get_victim_case_by_document", s.CurrentRole, c) {  
    return  
}
```

En el ejemplo anterior, antes de hacer cualquier cosa, se verifica si el usuario en sesión tiene permiso para acceder a la petición actual. Si no tiene permiso se retorna y no se hace nada.

Las fachadas sólo deben comunicarse con controladores.

Las fachadas tiene la capacidad de detectar si el servicio está siendo invocado por un cliente standalone o por un navegador y responder de acuerdo a esto, ya que en el caso del navegador debe responder con un HTML y en el caso de un cliente standalone responderá con un JSON. Aunque el primer request sea un HTML, internamente las acciones dentro de esa página serán llamados con Javascript en formato JSON.

```
if c.Request.Header["Accept"][0] == "application/json" {  
    c.DataFromReader(code, int64(len(victimCaseRes)), gin.MIMEJSON, strings.NewReader(victimCaseRes), nil)  
} else {
```

En el anterior código se ve la verificación mencionada; si viene en el header del HTTP Request la indicación de que se acepta "application/json", entonces se escribe directamente la respuesta.

En caso contrario, se deben enviar todos los parámetros requeridos por la plantilla HTML:

```
common_routers.RenderTemplate(c, salvia_daos.VictimContactEntityName, "salvia", "victim_case/", salvia_config.HTML_Templates, "report_victim_cases",  
    map[string]interface{}{  
        "currentUser":      s.Names + " " + s.LastNames,  
        "nav_rules":         salvia_config.TranslateNavigationRule(s.Lang, salvia_config.NAVIGATION_RULES["report_victim_cases"]),  
        "locale":            salvia_config.Locale,  
        "lang":              s.Lang,  
        "menu":              menu,  
        "departments":      departments,  
        "docType":           common_config.DOCUMENT_TYPE,  
        "violenceExperienced": salvia_config.VIOLENCE_EXPERIENCED,  
        "caseStatus":        salvia_config.VICTIM_CASE_STATUS[s.Lang],  
        "salviaAlertFormPath": salvia_config.FormPaths[s.Lang]["AlertGET"],  
        "securityCityFormPath": security_config.FormPaths[s.Lang]["CityGET"],  
        "securityTownFormPath": security_config.FormPaths[s.Lang]["TownGET"],  
    }, utils.GetFullHtmlFuncMap())
```

En el ejemplo anterior, RenderTemplate se encarga de renderizar la plantilla HTML en donde se le pasan muchas variables necesarias. Hay unas que son comunes:

currentUser: se encuentran los datos del usuario en sesión para poder mostrar sus nombres en la interfaz gráfica

nav_rules: contiene las reglas de navegación. Se puede definir que para cada acción en el formulario, la aplicación pueda redirigirse a diferentes partes. Por ejemplo, si se crea un caso nuevo, allí se especifica que debe redirigirse a listar los casos.

Router

El router es el encargado de definir los puntos de entrada, cada módulo tiene un MainRouter:

```
/*
|   VictimCase
*/
translatedEntity = salvia_config.Locale["sp"][salvia_daos.VictimCaseEntityName]

secRouter.POST("/"+translatedEntity, VictimCasePOST)
secRouter.POST("/"+translatedEntity+"/:id", VictimCasePOST)
secRouter.GET("/"+translatedEntity+"/:id/"+translatedNew, VictimCasePOST_GET)
secRouter.GET("/"+translatedEntity+"/"+translatedNew, VictimCasePOST_GET)

secRouter.GET("/"+translatedEntity, VictimCaseGET)
secRouter.GET("/"+translatedEntity+"/:id", VictimCaseGET)
secRouter.GET("/"+translatedEntity+"/:id/", VictimCaseGET)
secRouter.GET("/"+translatedEntity+"/:id/"+translatedDocument+"/:docType", VictimCaseGET)
secRouter.PUT("/"+translatedEntity, VictimCasePUT)

secRouter.PUT("/"+translatedEntity+"/:id/:by", VictimCasePUT)

secRouter.GET("/"+translatedEntity+"/"+translatedReport, VictimCaseReportGET)
secRouter.POST("/"+translatedEntity+"/"+translatedReport, VictimCaseReportPOST)
```

Como se puede observar, está la lista de todas las funcionalidades disponibles para VictimCase, de acuerdo al método HTTP y la ruta con su estructura, acompañada de la función que debe atender esa ruta. Esa función es una función de la fachada que recibe la petición directamente. La variable “translatedEntity” traduce el nombre de la entidad al idioma especificado para que el router configure una URL en dicho idioma.

Guía de configuración e instalación en diferentes entornos

La aplicación inicia en **Main.go** en la raíz y utiliza la carpeta **certs** para cargar los certificados SSL. Abre el puerto 80 y 443. Todo el tráfico por el puerto 80 se redirigirá al 443 para obtener el nivel de seguridad SSL.

En la raíz también se encuentra el directorio **config** en cuyo interior se aloja **db_config.json** que contiene la información de conexión a la base de datos del sistema.

Config (Por cada módulo)

Cada módulo a su vez tiene el directorio **config** que contiene tres archivos importantes:

Enums.go: contiene las enumeraciones de datos que son estáticos, como tipos de documentos, códigos de los

roles, y cosas típicas del modelo como los tipos de agresión, etnicidad, etc.

Locale.go: contiene todas las traducciones del sistema, tanto las de URL, como de labels y enums. La aplicación está diseñada para soportar múltiples idiomas, sin embargo por el momento sólo está el español y requerirá de terminar ajustes ya que algunas partes de las plantillas o del sistema está ligado al español.

Menu.go: contiene información sobre los menús para que sean dinámicos y por rol, aunque en salvia no se utiliza mucho, pero también contiene los permisos por roles.

Config.go: contiene información sobre configuración general, en este caso datos del dominio actual, y datos del servidor de correo que utilizará el sistema para el envío de notificaciones.

Config (En el directorio raíz)

El directorio **config** contiene un archivo principal llamado **db_config.json** que contiene las credenciales de acceso a la base de datos:

Despliegue del sistema en diferentes entornos

Para compilar la aplicación para linux y windows desde windows, se recomienda usar el siguiente comando:

```
$Env:GOOS="linux"; $Env:GOARCH="amd64"; go build -o salvia
```

```
$Env:GOOS="windows"; $Env:GOARCH="amd64"; go build -o salvia.exe
```

Para compilar la aplicación para linux y windows desde linux, se recomienda usar el siguiente comando:

```
GOOS=linux GOARCH=amd64 go build -o salvia
```

```
GOOS=windows GOARCH=amd64 go build -o salvia.exe
```

Tanto en linux como windows se requieren los siguientes archivos y directorios para que el sistema salvia funcione:

- certs/
 - salvia.crt
 - salvia.Key
- salvia.exe (o **salvia** si es en linux)

Toda la aplicación y archivos están autocontenidos en el binario, exceptuando los certificados.

En linux se recomienda ejecutar con el siguiente comando:

```
./salvia > logs.txt 2>&1 &
```

En donde todos los logs y salidas del sistema quedarán almacenados en logs.txt.

En windows se recomienda usar el TaskScheduler para ejecutar salvia.bat con el siguiente contenido:

```
@echo off
```

```
cd C:\salvia
```

```
.\salvia.exe > logs.txt 2>&1
```

Recomendaciones para Mantener la Integridad del Sistema

1. Introducción

Este capítulo tiene como objetivo establecer las recomendaciones y buenas prácticas que se deben seguir para mantener la integridad, disponibilidad y confidencialidad de la plataforma diseñada para el registro de violencias basadas en género actualmente desplegada en un servidor RedHat. Se detallan aquí las prácticas recomendadas para asegurar la continuidad y robustez del sistema.

2. Objetivos

- **Garantizar la integridad del sistema:** Asegurar que los datos y procesos no sean alterados de forma no autorizada.
- **Mantener la disponibilidad:** Minimizar interrupciones en el servicio y asegurar una respuesta rápida ante incidentes.
- **Proteger la confidencialidad:** Salvaguardar la información sensible y la identidad de los usuarios.
- **Facilitar la gestión de vulnerabilidades:** Establecer procesos para la detección, análisis y corrección de nuevas vulnerabilidades.
- **Mejorar la resiliencia ante incidentes:** Definir protocolos de respuesta y recuperación ante situaciones de emergencia.

3. Recomendaciones Generales de Seguridad

3.1. Actualizaciones y Gestión de Parches

- **Actualización del Sistema Operativo y Software:**
 - Mantener actualizado el servidor RedHat y todos los componentes del sistema con los parches de seguridad más recientes.
 - Configurar actualizaciones automáticas o programadas para minimizar la ventana de exposición a vulnerabilidades conocidas.
- **Actualización de las Aplicaciones Móviles:**
 - Publicar actualizaciones periódicas para las versiones de Android e iOS, corrigiendo vulnerabilidades y mejorando la seguridad (por ejemplo, actualizaciones de Flutter y dependencias asociadas).

3.2. Control de Accesos y Autenticación

- **Políticas de Autenticación Fuertes:**
 - Implementar autenticación multifactor (MFA) para los usuarios administrativos y operativos del sistema.
 - Utilizar contraseñas robustas y únicas, fomentando el uso de gestores de contraseñas para evitar la reutilización.
- **Gestión de Roles y Permisos:**
 - Definir roles y aplicar el principio de “mínimo privilegio”, asegurando que cada usuario y operador tenga acceso únicamente a las funciones necesarias para su función.
 - Revisar y actualizar periódicamente los permisos de acceso, especialmente tras cambios en el personal.

3.3. Encriptación de Datos

- **Datos en Tránsito:**
 - Garantizar que todas las comunicaciones entre clientes (navegadores y apps móviles) y el servidor se realicen a través de HTTPS con certificados SSL/TLS válidos.
- **Datos en Reposo:**
 - Implementar mecanismos de cifrado para bases de datos y archivos sensibles, de modo que la información almacenada esté protegida ante accesos no autorizados. En el caso del sistema Salvia, los datos en reposo están protegidos por las políticas de acceso y seguridad configurados en RedHat.

3.4. Monitoreo y Registro (Logging)

- **Registro de Eventos y Auditorías:**
 - Configurar un sistema de logs centralizado que capture eventos críticos (autenticación, cambios en la configuración, accesos anómalos, etc.).
 - Implementar alertas en tiempo real que notifiquen al equipo de seguridad sobre actividades sospechosas o incidentes.
- **Revisión Periódica de Logs:**
 - Establecer procedimientos para el análisis regular de los logs y auditorías internas que permitan identificar y corregir comportamientos inusuales.

3.5. Escaneo de Vulnerabilidades y Pruebas de Seguridad

- **Evaluación Continua:**
 - Realizar pruebas de penetración (pentesting) y análisis de vulnerabilidades de forma periódica, siguiendo marcos como el OWASP Top 10.
- **Gestión de Dependencias:**
 - Supervisar y actualizar las bibliotecas y componentes de terceros para evitar el uso de componentes vulnerables o desactualizados. EN el caso de Salvia, no hay dependencias externas, más allá de PostgreSQL

3.6. Respaldo y Recuperación

- **Políticas de Backup:**
 - Establecer una estrategia de respaldo regular de la base de datos y configuraciones críticas, asegurando que se realicen copias de seguridad en ubicaciones seguras.
 - Probar los procedimientos de recuperación de datos de forma periódica para garantizar la continuidad del servicio ante incidentes.

3.7. Gestión de Incidentes y Respuesta ante Emergencias

- **Plan de Respuesta a Incidentes:**
 - Definir y documentar un protocolo de respuesta a incidentes de seguridad, que incluya identificación, contención, erradicación, recuperación y lecciones aprendidas.
 - Capacitar al personal en la ejecución de este plan y realizar simulacros de incidentes.
- **Coordinación con Equipos de Seguridad:**
 - Mantener un canal de comunicación abierto con el equipo de ciberseguridad y, en caso necesario, con las autoridades pertinentes para una respuesta rápida.

4. Recomendaciones Específicas para Componentes del Sistema

4.1. Servidor (RedHat)

- Configurar firewalls (tanto de red como de aplicaciones web – WAF) para filtrar tráfico malicioso.
- Asegurar la configuración de servicios y puertos, limitando el acceso a solo aquellos necesarios para el funcionamiento.
- Implementar mecanismos de detección y prevención de intrusiones (IDS/IPS).

4.2. Aplicación Web (Backend en Go y HTML)

- Revisar y reforzar la validación y sanitización de todas las entradas para prevenir ataques de inyección (SQL, XSS, etc.).
- Aplicar principios de programación segura y realizar code reviews para detectar vulnerabilidades en el código fuente.
- Utilizar bibliotecas y frameworks que cuenten con soporte y actualizaciones regulares de seguridad.

4.3. Aplicaciones Móviles (Cliente en Flutter)

- Implementar medidas de seguridad específicas en el código móvil, como el uso de almacenamiento seguro (por ejemplo, flutter_secure_storage) y autenticación biométrica.
- Asegurar la comunicación con el servidor mediante HTTPS y manejar adecuadamente los tokens y claves de sesión.

5. Recomendaciones Organizacionales y de Proceso

- **Capacitación Continua:**
 - Organizar sesiones de formación y talleres en ciberseguridad para el equipo de desarrollo y el personal operativo.
- **Políticas y Procedimientos:**
 - Establecer y documentar políticas de seguridad de la información, incluyendo procedimientos para la gestión de accesos, incidentes y cambios en la infraestructura.
- **Revisión y Actualización del Plan:**
 - Programar revisiones periódicas del plan de seguridad y actualización del documento de recomendaciones conforme evolucionen las amenazas y se introduzcan nuevas tecnologías o metodologías.

6. Conclusiones

Adoptar estas recomendaciones permitirá mantener la integridad del sistema a lo largo del tiempo, mitigando el riesgo de vulnerabilidades y asegurando la protección de datos sensibles. La combinación de medidas técnicas, organizacionales y de procesos crea una defensa en profundidad que es fundamental para una plataforma crítica como la que se utiliza para reportar casos de violencia basada en género.

La implementación de estas prácticas y el compromiso continuo con la actualización y capacitación en ciberseguridad garantizarán que el sistema se mantenga robusto, resiliente y confiable, brindando confianza tanto a los usuarios que reportan casos como a los operadores y funcionarios encargados de la atención.